CSCI 232 Assignment 06 – Doubly Linked List: Your Own vs. the Standard Library

Points: 10

Instructions

1. **Fork** the repository on GitHub: `https://github.com/jpach-cs/232-ass06-template`
2. **Clone** your forked repository to your local machine.
3. Open `src/code.cpp`, `src/tests.cpp` and `include/code.hpp`.
4. Uncomment the author block in `src/code.cpp` and fill in your name:

```
// std::string AUTHOR_NAME       = "Your Name";
// std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this
// assignment independently...";
```

5. Work through each class in **four parts**:

Part 1 — Class `DoubleNode` (`src/code.cpp`)

- Implement the constructor (member initializer list), three `const` getters and two setters.
- A node knows nothing about the list: it does not print, search or delete.

Part 2 — Class `DoublyLinkedList` (`src/code.cpp`)

- The list **owns** its nodes (composition): every `new` and `delete` happens inside the class.
- Implement the three private `detach*` building blocks first, then build the public methods on top of them.
- Implement RAII memory management in the constructor and destructor.
- Ensure all methods handle the empty list, the one-node list, and `nullptr` guards.

Part 3 — Class `StdDoublyLinkedList` (`src/code.cpp`)

- Do **not** build a list. Build a thin **wrapper** around one `std::list<NodeValue>` that behaves exactly like `DoublyLinkedList`.
- Every method body is only a few lines of library calls. Read cppreference.com to find the right tool.

Part 4 — Unit Tests (`src/tests.cpp`)

- Implement the 8 unit test bodies using `TEST_ASSERT_`.

1. Build and run locally using C++20 (`g++`):

```
make clean
make
./run_tests
./check_rules.sh
```

7. When all local tests pass — **commit** and **push** to your repository.
8. Go to the **Actions** tab on GitHub and verify the CI results.

Class `DoubleNode` — `src/code.cpp` (Part 1)

A node is a **class**, not a `struct`. Every field is declared **separately**, with its own access modifier. The class is declared in `include/code.hpp`:

```
class DoubleNode
{
   private: NodeValue   value;
   private: DoubleNode *prevPtr;
   private: DoubleNode *nextPtr;

   public:  explicit DoubleNode(NodeValue newValue);

   public:  NodeValue   getValue() const;
   public:  DoubleNode *getPrev() const;
   public:  DoubleNode *getNext() const;

   public:  void setPrev(DoubleNode *newPrevPtr);
   public:  void setNext(DoubleNode *newNextPtr);

   // given: copying a node is forbidden (explained in a later lecture)
   public:  DoubleNode(const DoubleNode &) = delete;
   public:  DoubleNode &operator=(const DoubleNode &) = delete;
};
```

Method	Description
`DoubleNode(NodeValue)`	Constructor — initialize **all three fields** with an initializer list: `value` gets `newValue`, `prevPtr` and `nextPtr` are `nullptr`
`getValue`	`const` getter — return the stored value
`getPrev`	`const` getter — return the previous node (or `nullptr`)
`getNext`	`const` getter — return the next node (or `nullptr`)
`setPrev`	Store the given pointer in `prevPtr`
`setNext`	Store the given pointer in `nextPtr`

Reading the constructor declaration

```
explicit DoubleNode(NodeValue newValue);
//                  ^^^^^^^^^ ^^^^^^^^
//                  type      parameter name
```

- `NodeValue` is the type `std::variant<int, double, std::string>`. The node can hold an `int`, a `double` or a `std::string`.
- `newValue` is passed **by value**, so the constructor receives its own copy.
- The parameter is called `newValue`, not `value`, because the field is already called `value`. Two different things must not share one name.
- In the `.cpp` the definition starts with the class name and `::`

```
DoubleNode::DoubleNode(NodeValue newValue)
    : /* TODO: initialize all three fields */
{
}
```

Calling it: `new DoubleNode(NodeValue{5})`, `new DoubleNode(NodeValue{2.5})`, `new DoubleNode(NodeValue{std::string("abc")})`.

`DoublyLinkedList` Class Methods — `src/code.cpp` (Part 2)

The list **owns** its nodes (composition).

> The outside world works only with **values**. No public method may take, return or expose a `DoubleNode*`. Every `new` and every `delete` of a node happens **inside** the class.

Private methods

The caller of a `detach*` method owns the returned node and must `delete` it. A `detach*` method never deletes anything itself.

Method	Description
`detachFirst`	Unlink the first node and **return it** (do not delete). Fix `headPtr`, `tailPtr`, `count`. Clear the node's own `prevPtr` and `nextPtr`. Return `nullptr` if the list is empty
`detachLast`	Same for the last node. Use the tail's `prev` link, so no walking is needed
`detachValue`	Find the **first** node equal to `value` and unlink it (head, tail, only node or middle node). Return `nullptr` if not found. Reuse `detachFirst` and `detachLast` where you can

Public methods

Method	Description
`DoublyLinkedList()`	Constructor — initialize empty list (`headPtr = nullptr`, `tailPtr = nullptr`, `count = 0`)
`~DoublyLinkedList()`	Destructor — delete **all** nodes (RAII). Save the next pointer **before** `delete`
`pushFrontValue`	Create a node with `new` and link it at the front. Handle the **empty list** (the new node is both head and tail)
`pushBackValue`	Create a node with `new` and link it at the end. Handle the **empty list**
`popFrontValue`	Detach the first node, read its value, `delete` the node, return the value. Empty list → `std::nullopt`
`popBackValue`	Detach the last node, read its value, `delete` the node, return the value. Empty list → `std::nullopt`
`deleteValue`	Remove and free the **first** node equal to `value`. Return `true` if something was removed
`containsValue`	Return `true` if any node holds `value`
`printForward`	Print values head → tail to `std::cout` using `std::visit` (each value followed by a space, then a newline). Return the number of printed values
`printBackward`	Print values tail → head using `prevPtr`. Return the number of printed values
`length`	Return the current number of nodes (`count`)

Using `std::optional`

`std::optional<NodeValue>` is a box that holds **either a `NodeValue` or nothing**. It is a sibling of `std::variant`: with `variant` you check **which type** is inside, with `optional` you check **whether anything** is inside.

```
#include <optional>

std::optional<NodeValue> full = NodeValue{5};   // a value was assigned
std::optional<NodeValue> empty;                 // nothing was assigned

full.has_value();     // true
empty.has_value();    // false

full.value();         // NodeValue{5}, the value inside the box
```

- `empty = std::nullopt;` puts the box back to the "nothing" state.

- Check first, then open the box:

```
if (full.has_value())
{
    NodeValue v = full.value();
}
```

> Calling `.value()` on an empty `optional` throws an exception and stops the program. Always ask `has_value()` first, unless you are certain a value is there.

In this assignment: a function whose return type is `std::optional<NodeValue>` does one of two things.

- It has a value: `return value;` (the box is full).
- It has nothing: `return std::nullopt;` (the box is empty).

Watch out

- `NodeValue{1}` and `NodeValue{1.0}` are **not equal**: `variant` compares the active type first, then the value.
- The fields of `DoubleNode` are private. The list reads and changes links only through `getPrev`, `getNext`, `setPrev` and `setNext`.
- Deleting the only node must set **both** `headPtr` and `tailPtr` to `nullptr`.
- Removing the head: the new head's previous link must become `nullptr`. Removing the tail: the new tail's next link must become `nullptr`.
- Removing a middle node needs **two** link repairs. Forget one and `printBackward` or `popBackValue` breaks later, far from the real bug.
- Never read a node after `delete`. Read what you need (`getNext()`, `getValue()`) first.

`StdDoublyLinkedList` Class Methods — `src/code.cpp` (Part 3)

```
Outside:   StdDoublyLinkedList   looks the same as   DoublyLinkedList
Inside:    one std::list                      vs.    your own nodes and pointers
```

The caller must not be able to tell the two classes apart: the same calls, the same results. The class has **one private field** (`std::list<NodeValue> data`) and the **same public methods** as Part 2. There is **no** constructor, destructor, `detach*` or `= delete`: `std::list` cleans up after itself, the compiler-generated versions are enough.

Method	The question you must answer
`pushFrontValue`, `pushBackValue`	Which `std::list` methods insert at either end?

Method	The question you must answer
`popFrontValue`, `popBackValue`	How do you read the end element and remove it? What if the list is empty?
`deleteValue`	How do you remove only the **first** element equal to `value`?
`containsValue`	How do you search without writing a loop?
`printForward`, `printBackward`	How do you go through the list from the tail?
`length`	Which method gives the size, and which type does it return?

Useful starting points on cppreference.com:

- `std::list`: `push_front`, `push_back`, `front`, `back`, `pop_front`, `pop_back`, `empty`, `size`, `begin`, `end`, `rbegin`, `rend`, `erase`, `remove`
- `<algorithm>`: `std::find`

Hints

- Some names look like they do the same thing as your methods. **Read the documentation, not just the name.** Two of the tools behave differently from what you need. Your tests will tell you which.
- Some `std::list` methods are **undefined behavior** on an empty list. Your public methods must still behave exactly as in Part 2.
- `size()` returns `size_t`, but `length()` returns `int`. Use `static_cast<int>(...)`.
- For printing, use `std::visit` as in `printList` from Assignment 05. Take the value **by copy**: `[](auto x) { std::cout << x << " "; }`.

  Same interface, different inside. That is what an abstract data type is. Compare the size of Part 3 with Part 2: this is how much work the library does for you.

Tests to implement — `src/tests.cpp` (Part 4)

They are not meant to be exhaustive. Their purpose is to convince **you** that you handled the edge cases. The instructor tests in CI check correctness in depth, and they run the **same** suite on both classes.

Test function	What to verify
`test_new_list_is_empty`	`length()` is `0` on a new `DoublyLinkedList`
`test_push_increases_length`	After `pushFrontValue` and `pushBackValue`, `length()` is `2`

Test function	What to verify
`test_pop_on_empty_list`	`popFrontValue()` and `popBackValue()` on an empty list have no value
`test_pop_order`	Push `1, 2, 3` with `pushBackValue`: `popBackValue()` gives `3`, `popFrontValue()` gives `1`, `length()` is `1`
`test_delete_value_not_found`	`deleteValue` with a missing value returns `false` and `length()` does not change
`test_delete_middle_keeps_links`	Push `1, 2, 3`, delete `2`: `popBackValue()` gives `3`, the next `popBackValue()` gives `1`
`test_std_delete_only_first`	`StdDoublyLinkedList`: push `1, 2, 1`, `deleteValue(NodeValue{1})`: `length()` is `2` and `popFrontValue()` gives `2`
`test_std_pop_on_empty_list`	`StdDoublyLinkedList`: `popFrontValue()` and `popBackValue()` on an empty list have no value and do not crash

`test_delete_middle_keeps_links` is the most important one: `popBackValue` follows `prevPtr`, so it fails if you forgot to repair the backward link.

Example:

```
void test_pop_order(void)
{
    DoublyLinkedList list;
    list.pushBackValue(NodeValue{1});
    list.pushBackValue(NodeValue{2});
    list.pushBackValue(NodeValue{3});

    TEST_ASSERT_TRUE_MESSAGE(list.popBackValue().value() == NodeValue{3}, "popBackValue shou
    TEST_ASSERT_TRUE_MESSAGE(list.popFrontValue().value() == NodeValue{1}, "popFrontValue sh
    TEST_ASSERT_TRUE_MESSAGE(list.length() == 1, "Length should be 1");
}

void test_std_pop_on_empty_list(void)
{
    StdDoublyLinkedList list;

    TEST_ASSERT_TRUE_MESSAGE(!list.popFrontValue().has_value(), "popFrontValue on empty list
```

```
    TEST_ASSERT_TRUE_MESSAGE(!list.popBackValue().has_value(), "popBackValue on empty list r
}
```

Rules

- Do **not** modify `include/code.hpp` or any function **signature**.
- All code must compile under **C++20** (`-std=c++20`).
- No public method of `DoublyLinkedList` may take, return or expose a `DoubleNode*`.
- Always guard against `nullptr` before dereferencing a pointer.
- Always clean up dynamically allocated nodes (`delete`) exactly once, inside the class.
- The Part 3 section of `src/code.cpp` (between the `BEGIN PART 3` and `END PART 3` markers) must **not** contain `new`, `delete`, `DoubleNode` or `Ptr`, and no hand-written loop that **searches or removes**. A loop is allowed only for printing. Do not remove the markers. CI checks it (`check_rules.sh`).
- Call `.value()` on a `std::optional` only after checking `has_value()` (or when the test guarantees a value).
- Do **not** use references (`&`) or templates in your own code. We have not covered them yet. (The `= delete` lines in the given header and the `[](auto x)` printing lambda from the hint are the only exceptions.)
- CI builds with AddressSanitizer: memory leaks, use-after-free and `front()`/`back()` on an empty `std::list` fail the build.
- CI runs your `tests.cpp` together with the instructor tests — both must pass.

Suggested schedule

Week	Work
1	Part 1 and Part 2, plus your 6 tests for `DoublyLinkedList`
2	Part 3, your 2 tests for the wrapper

Part 3 is much shorter than Part 2, because you already know every operation.

Grading

Component	Points
`DoubleNode` class (Part 1)	1

Component	Points
`DoublyLinkedList` class implementation (RAII & methods)	4
`StdDoublyLinkedList` wrapper (Part 3, rules respected)	2
All test cases in `tests.cpp` passing	3
Total	**10**

Author block — required

```
std::string AUTHOR_NAME       = "Your Name";
std::string AUTHOR_AUTHORSHIP = "I acknowledge that I have worked on this "
"assignment independently, except where explicitly noted and referenced. "
"Any collaboration or use of external resources has been properly cited. "
"I am fully aware of the consequences of academic dishonesty and agree "
"to abide by the university's academic integrity policy.";
```